TOULOUSE INP N7

Projet détection d'objet - Apprentissage Profond

Détection Princesses

Aurore, Blanche-Neige, Raiponce

Base de Données :
https://git.inpt.fr/rabefanirakay/princess_guesser.git

Membres du groupe :

DISSOUBRAY Alice
DRAOU Iman-Norr
FLEURY Elena
LESCOT Alexandre
RABEFANIRAKA Yan

PARTIE 1

1. Description du sujet

Pour ce projet nous avons choisi de créer une base données d'images de princesses Disney. Pour cela, nous nous sommes focalisés sur trois princesses : *Aurore* de "la Belle au Bois Dormant", *Blanche-Neige* de "Blanche-Neige et les Septs Nains" et *Raiponce* de "Raiponce".

Ainsi pour notre jeu de données, nous avons des images tirées de ces films, des images de posters, et des images de princesses dérivées de ces films.

2. Description méthodologie pour acquérir et annoter

Pour constituer notre base de données nous avons récupéré nos images de 2 façons différentes :

1. Par un script python permettant de télécharger un grand nombre d'image depuis google
2. Une base de données contenant des images de princesses trouvée sur Kaggle https://www.kaggle.com/datasets/sayehkargari/princess-dataset?resource=download

Nous faisons en sorte de récupérer un grand nombre d'images avec ces deux sources afin d'avoir encore suffisamment de données après le tri.

Après un tri qui a permis d'enlever les images mal référencées ou trop dérivées des princesses, nous obtenons alors un dataset satisfaisant à nos yeux. Nous enlevons aussi les images qui ont pu être récupérées mais qui sont identiques. Cela n'est pas parfait parce qu'il reste encore des images très similaires. Après le tri, nous nous retrouvons avec entre 210 et 220 images pour chaque princesses ce qui nous fait une base de données de plus de 600 images, ce que nous considérons comme satisfaisant pour le projet. Nous apprécions aussi que nos 3 classes soient équilibrées en nombre d'images afin d'avoir un entraînement qui ne favorise pas une classe plus que les autres.

Ces 2 méthodes de récupération des données nous ont permis de nous donner des images variées par leur format, leur couleur, leur origine, et leur qualité. Les princesses portent différentes tenues, avec des arrière-plans divers, il y a des images où elles sont seules mais aussi accompagnées d'autres personnages, et même certaines ou plusieurs de nos 3 princesses sont dans la même image. Ainsi, nous évitons que le modèle n'apprenne qu'un seul type de situation.

Pour annoter les différentes images, nous avons utilisé comme outil *'LabelImg'* pour créer des boîtes englobantes au format YOLO. Nous avons choisi de créer des boîtes englobantes contenant la tête ainsi que le buste de la princesse en leur attribuant à chacune

une étiquette. Avec ces critères, nous avons chacun annoté une partie des images. On remarque que nos annotations ne sont pas exactes et varient d'une image à une autre pour un même annotateur et aussi entre deux personnes différentes.

3. Description partitionnement des données en train, test et val

Nous avons choisi d'utiliser 70% des données pour l'entraînement, le reste est séparé équitablement entre validation et test.

Pour répartir, nous faisons d'abord un tri aléatoire des images en respectant les proportions ci-dessus. Puis, nous faisons en sorte que dans les 3 catégories (train, test et val) chaque princesse soit représentée, et si elles ont différentes tenues, ces différentes tenues soient représentées dans ces 3 catégories pour éviter le fait qu'un type d'image ne soit présent que dans l'entraînement ou que dans la validation ou le test, ou une sur-représentation qui entraînerait un déséquilibre.

Nos classes étant équilibrées mais pas très grandes, cela ne sera peut-être pas suffisant pour un apprentissage satisfaisant. De plus, certaines images sont très similaires et cela pourrait mener vers du sur-apprentissage. Si nous avions plus de données et beaucoup plus de temps, il faudrait supprimer les images qui sont quasi-identiques pour avoir seulement des images réellement distinctes. Bien que cela pourrait réduire de façon non négligeable notre jeu de données.

Ainsi nous obtenons une telle répartition des images :

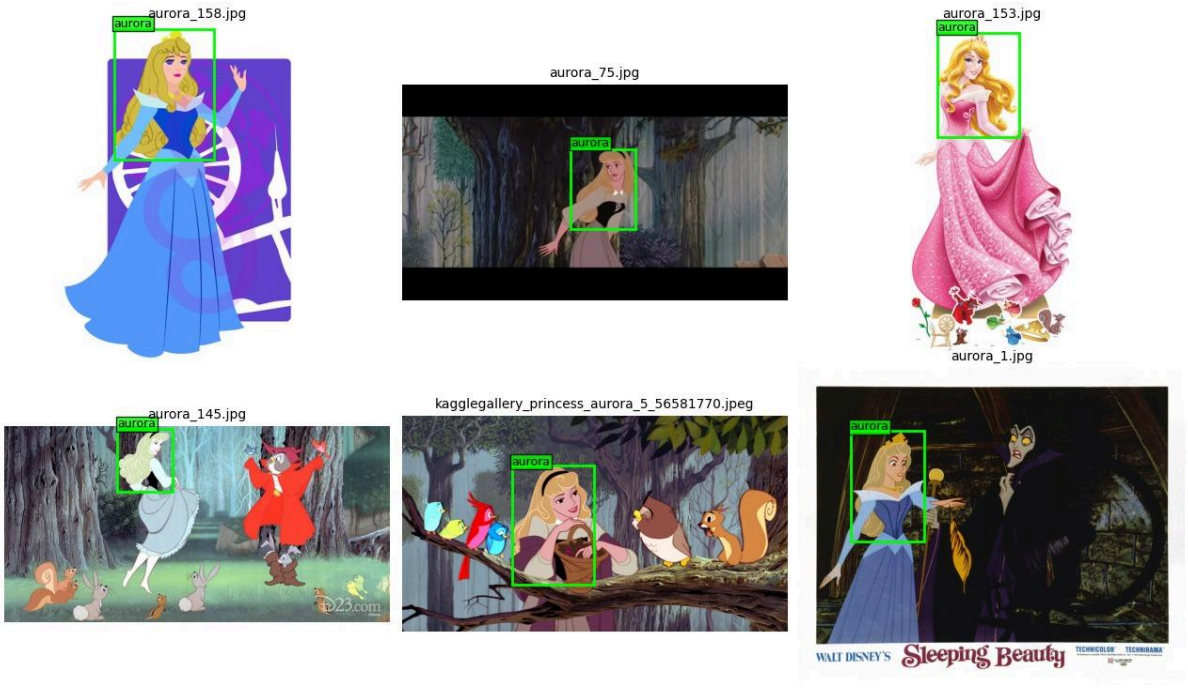
→ Aurore: total=212, train=148, val=32, test=32

→ Blanche Neige: total=223, train=156, val=33, test=34

→ Raiponce: total=218, train=153, val=33, test=32

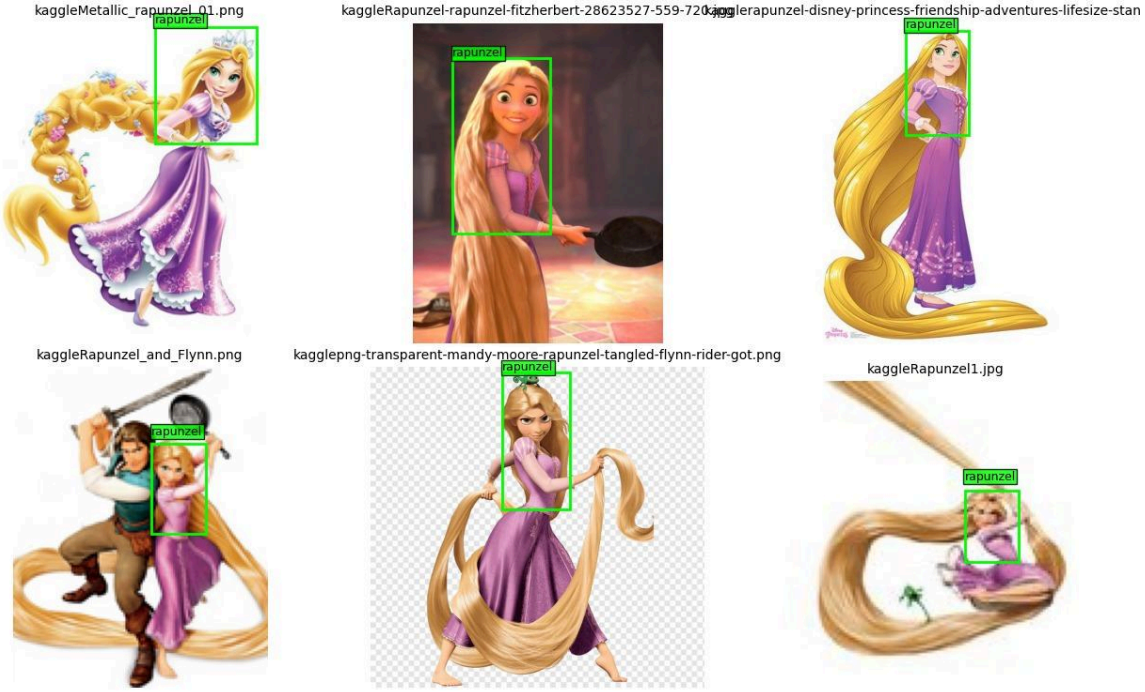
4. Exemples de la Base de Données

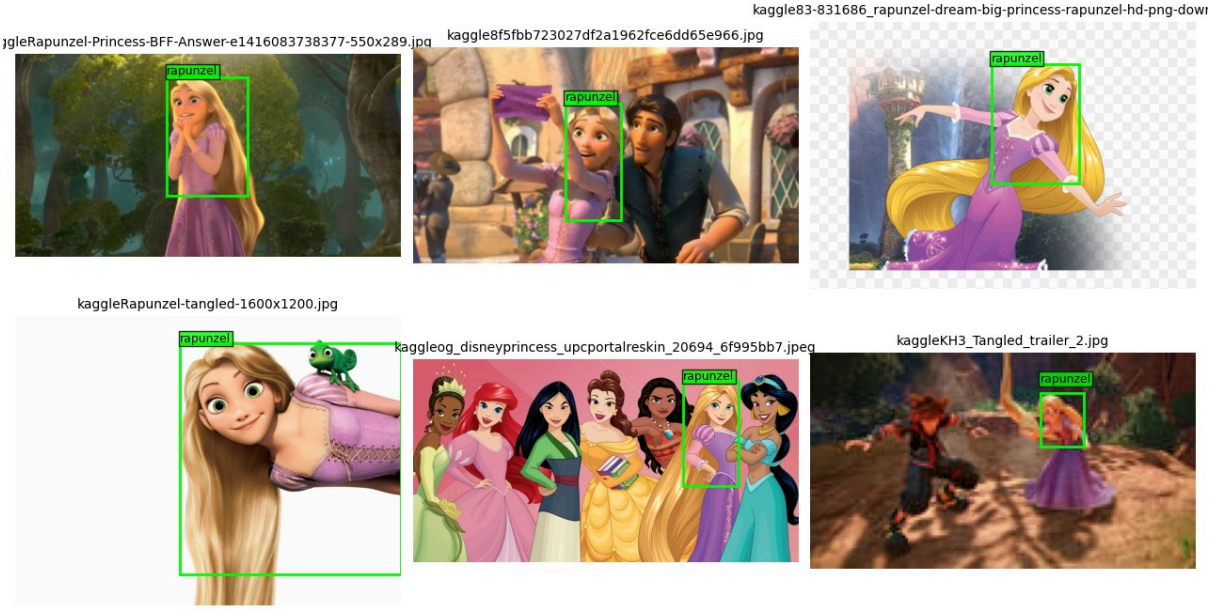
I. Aurore



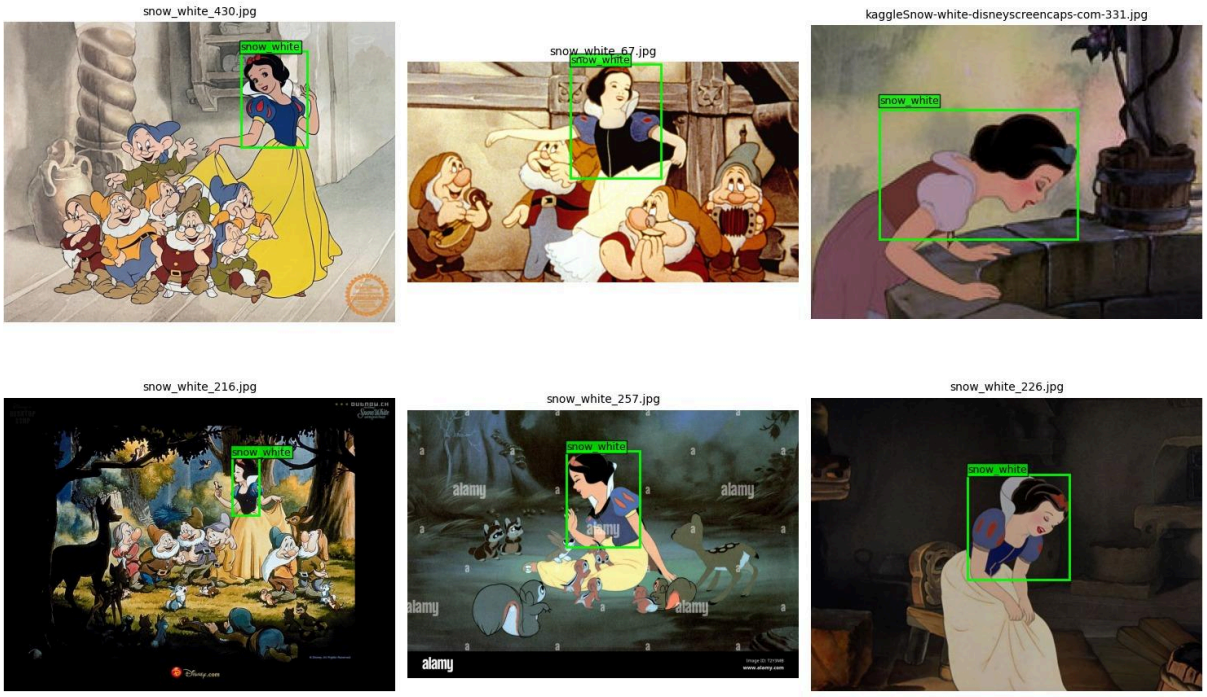

kaggledb1d3c9c539b1408b6787858c5783dde.jpg
aurora_10.jpg
aurora_31.jpg
kagglePrincess-Aurora-Blue.jpg
kaggleRBTJ_-_Aurora_concept.jpg
kagglepng-transparent-princess-aurora-belle-leia-organa-disney-pri.pr
aurora

II. Raiponce


kaggleMetallic_rapunzel_01.png
kaggleRapunzel-rapunzel-fitzherbert-28623527-559-720.jpg
kagglerapunzel-disney-princess-friendship-adventures-lifesize-stan
kaggleRapunzel_and_Flynn.png
kagglepng-transparent-mandy-moore-rapunzel-tangled-flynn-rider-got.png
kaggleRapunzel1.jpg
rapunzel



III. Blanche-neige



PARTIE 2

1) Préparation des données

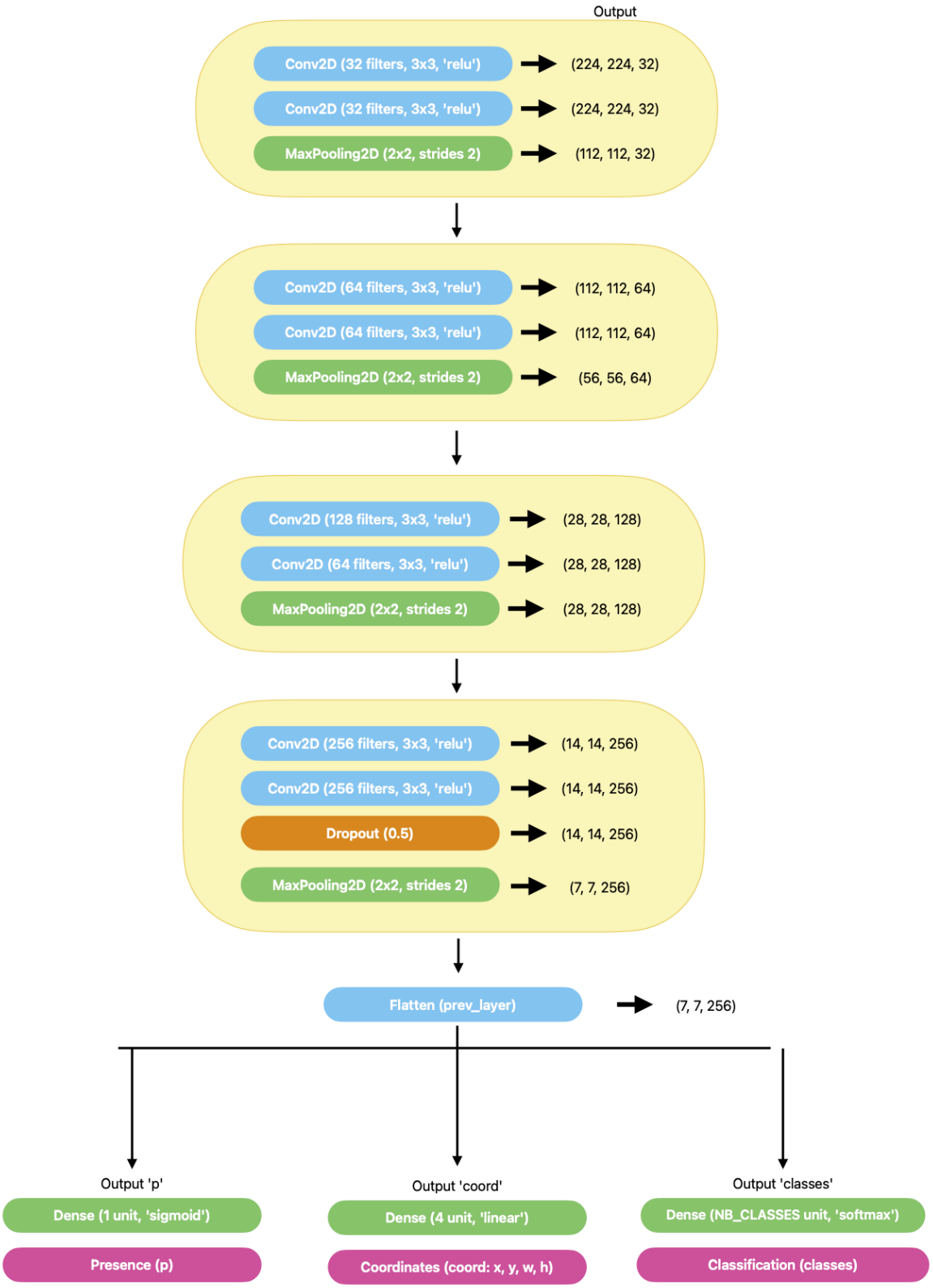
L’objectif est de détecter la présence d’un objet, localiser cet objet avec une boîte englobante et identifier sa classe.

On utilise *princess_guesser.csv* centralisant toutes nos informations. D’abord, on extrait le chemin de l'image, le nom de la princesse (qui est le label) et les coordonnées de la boîte englobante. Puis nous avons formaté les images (taille 64*64 avec trois canaux de couleur RGB). Chaque image génère un indicateur de présence, les coordonnées de la boîte converties, le nom de la princesse.

Pour améliorer l'entraînement, on a centré-réduit les coordonnées des bounding box.

Pour savoir si notre apprentissage est efficace, on utilise deux outils principaux présents dans le code : l'IoU et la Global Accuracy. Par exemple, pour l’IoU, si le score est supérieur à 0.4, on considère que l’apprentissage est bon.

2) Architecture de notre 1er modèle



C’est un réseau de neurones convolutifs avec sorties multiples.

Dans ce modèle nous avons **4 blocs de convolution successifs**. Chacun de ces blocs contient **deux couches de convolution**, *Conv2D*, et est suivi d’**une couche de sous-échantillonnage** *MaxPooling2D*.

Pour *Conv2D*, c’est l’activation ReLU qui est utilisée avec comme différents filtres 32, 64, 128, puis 256. La taille de noyau est 3×3, pour le padding "*same*" pour conserver les dimensions, et l’initialisation des poids avec "*he_normal*"

Le pooling (*MaxPooling2D*) pour la réduction de dimension de l’image permet de conserver l’essentiel de l’information. Un Dropout de 0.5 désactive la moitié des neurones pendant l'entraînement pour que le modèle n'apprenne pas les images par cœur, et ainsi limiter le surapprentissage.

La sortie est ensuite aplatie par une couche Flatten .

Nous avons 3 zones de sorties qui nous permettent de prédire : la présence d’une princesse (avec une activation sigmoïde) , les coordonnées de boîte englobante (une activation linéaire) et les probabilités de classe (activation softmax).

On utilise l’optimiseur Adam avec un taux d'apprentissage de 3*10^-4.

Nous avons 3 fonctions de perte : *BinaryCrossentropy* pour la présence et la classification, MeanSquaredError (MSE) pour la précision de la boîte englobante.

Le modèle est entraîné avec 50 epochs, des paquets de 16 images et un ensemble de validation.

On a aussi : loss_weights = [1,1,1] pour que chaque tâche ait la même importance pendant l’apprentissage.

Pour augmenter la diversité des images de notre entraînement nous procédons à une augmentation de données avec la bibliothèque *Albumentations*. Pour les transformations, nous avons utilisé *RandomGamma* et *RandomBrightnessContrast* avec une probabilité d’être appliquée de 50 % .

Nous utilisons le transfert d'apprentissage qui améliore également les résultats et limite le sur-apprentissage en réutilisant un réseau déjà entraîné sur une très grande base de données. Ici, le réseau est *VGG16*, qui est préentraîné sur la base de données *ImageNet*.

3) Ultralytics

Dans cette dernière partie, nous avons utilisé *Ultralytics* avec *YOLO.*

Pour l'entraînement du modèle :

- Nombre d'époques : 100
- Taille des images : 64
- Dataset : princesses.yaml

Nous avons toujours les 3 classes *'Snow_white'*, *'Aurora'* et *'Rapunzel'* .

On conserve le modèle correspondant aux meilleurs poids . On sélectionne 10 images au hasard. Pour l'inférence, les prédictions sont réalisées avec *imgsz=64* (la taille des images à l'entrée) et *conf=0.25* (le de confiance seuil minimal)

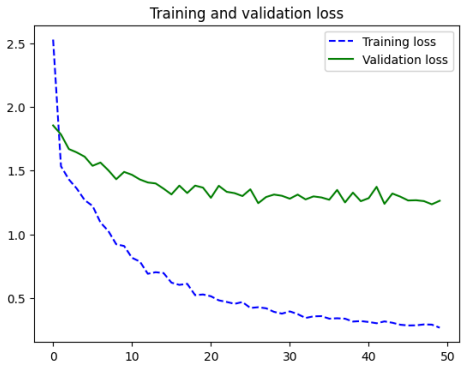
PARTIE 3

1) Analyse quantitative

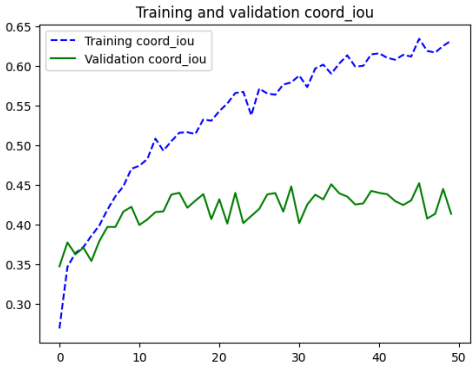
Pour le modèle entraîné avec les fonctions de coût suivantes :

- la détection de présence ou non d'une princesse —> Binary Cross Entropy
- les coordonnées de la boîte englobante —> Mean Square Error
- la classification de choix entre les 3 princesses —> Binary Cross Entropy

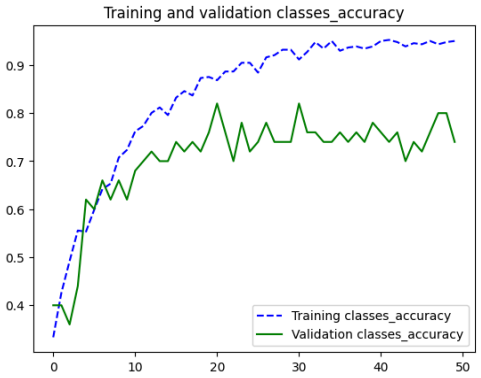
Ci-dessous l'évolution de la fonction de coût pour la détection, de l'IoU pour mesurer la qualité des boîtes englobantes et la fonction coût pour la classification :



Fonction de coût détection



IoU_qualité des boîtes englobantes



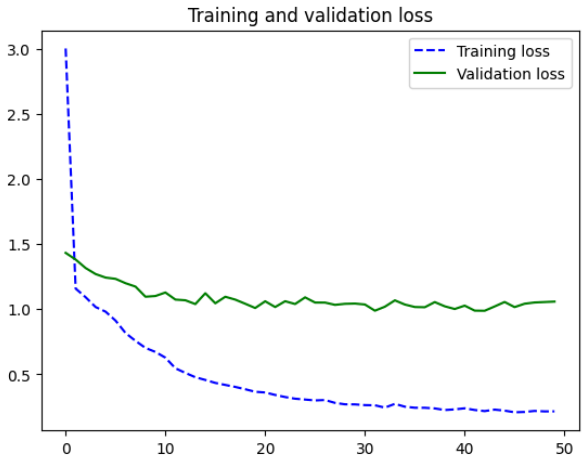
Fonction de précision classification

On obtient une précision globale peu satisfaisante:

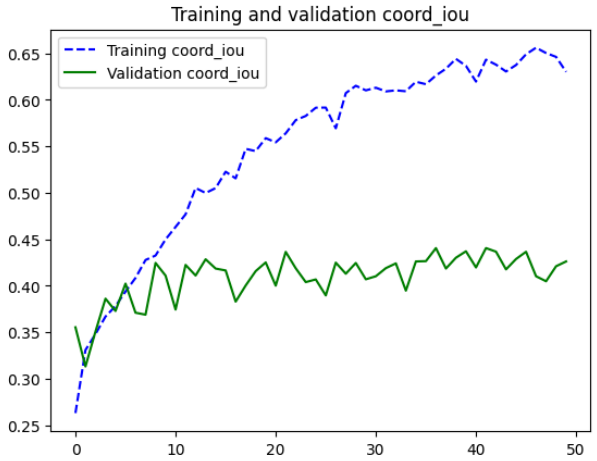
La précision globale est de 26.0%

Classe	Précision	Rappel	F1-score
Raiponce	0.14	0.18	0.16
Blanche Neige	0.36	0.38	0.38
Aurore	0.33	0.22	0.26

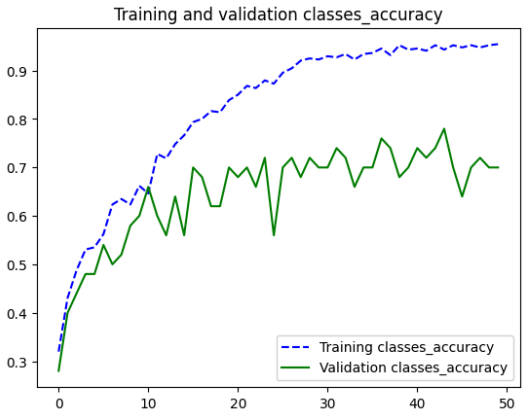
En utilisant seulement l'erreur quadratique moyenne comme fonction de coût, on obtient les résultats suivants :



Fonction de coût détection



IoU_qualité des boîtes englobantes

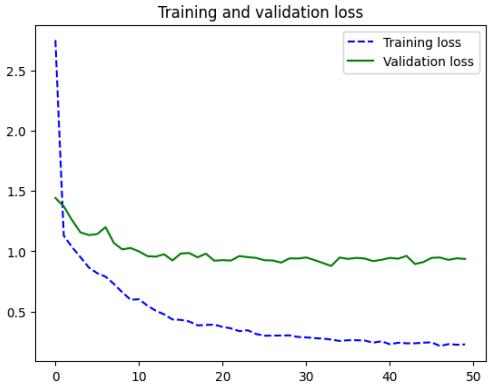


Fonction de précision classification

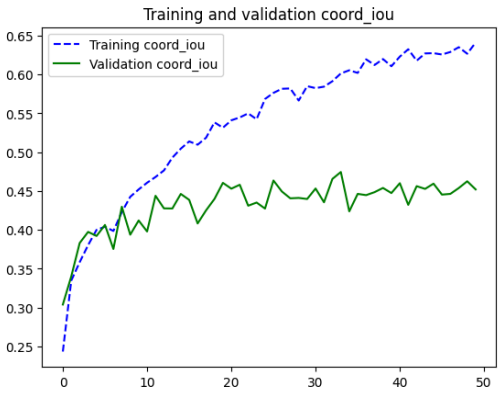
La précision globale est de 35.8%, on commence à améliorer nos performances

Classe	Précision	Rappel	F1-score
Raiponce	0.27	0.21	0.24
Blanche Neige	0.41	0.49	0.44
Aurore	0.36	0.37	0.37

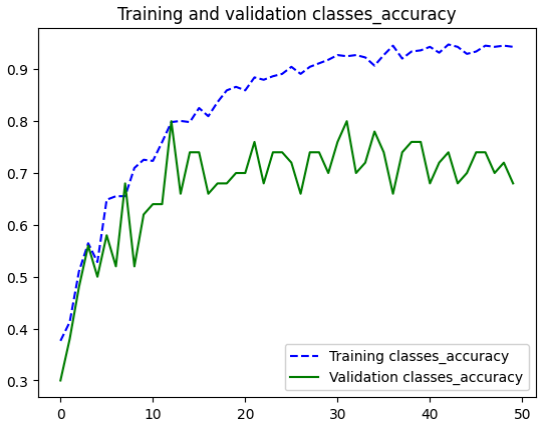
Après l'augmentation des données on obtient cette fois-ci ces résultats :



Fonction de coût détection



IoU_qualité des boîtes englobantes

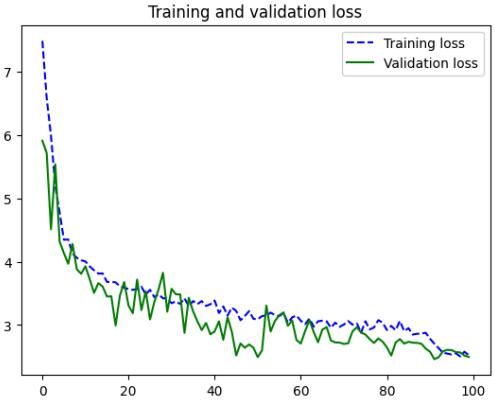


Fonction de précision classification

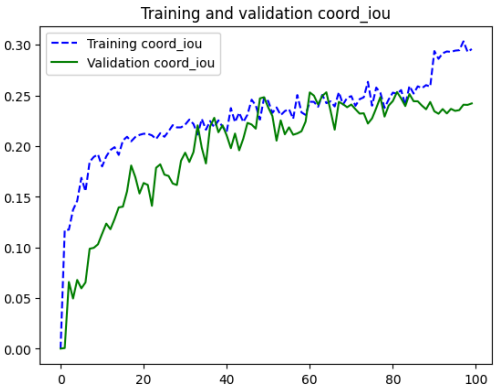
La précision globale est de 40.2%, on devient encore meilleur ici !

Classe	Précision	Rappel	F1-score
Raiponce	0.39	0.34	0.37
Blanche Neige	0.41	0.54	0.47
Aurore	0.40	0.31	0.35

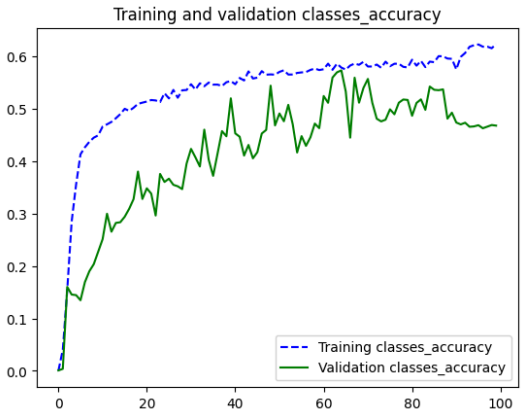
Finalement, avec le transfert d'apprentissage avec *Ultralytics*:



Fonction de coût détection



IoU_qualité des boîtes englobantes



Fonction de précision classification

La précision globale est de 72.4% une performance qui double presque toutes les autres méthodes !

Classe	Précision	Rappel	F1-score
Raiponce	0.71	0.60	0.65
Blanche Neige	0.74	0.66	0.69
Aurore	0.76	0.69	0.72

2) Analyse qualitative

La première chose que l'on peut noter c'est que l'on arrive avec nos modèles à détecter et classifier nos 3 princesses notamment dans des cas favorables, c'est-à-dire pour des images simples, lorsque les princesses sont bien visibles et assez grandes sur l'image. Dans ces conditions, la boîte englobante est bien positionnée et la classification est correcte, ce qui montre que le réseau a bien appris les caractéristiques visuelles principales de chaque princesse.

Les meilleurs résultats sont obtenus pour Blanche-Neige, ce qui est logique : ses caractéristiques visuelles sont très distinctives (cheveux noirs courts, robe aux couleurs très contrastées) et la différencient clairement des deux autres princesses. Aurore et Raiponce partagent des caractéristiques similaires (longs cheveux, robes aux teintes pastel) ce qui explique les confusions fréquentes entre ces deux classes, notamment à faible résolution.

Le choix de prendre comme résolution d'entrée 64 pixels par 64 est aussi une source de limitation puisqu'avec une résolution aussi faible on perd nécessairement des détails et il devient donc plus difficile de récupérer et trouver des détails caractéristiques et marquants qui permettent de trouver les princesses.

On remarque également des erreurs lorsque plusieurs personnages sont présents sur la même image, ce qui perturbe la détection, ou lorsque la princesse est petite ou partiellement visible. Ces cas sont aussi peu représentés dans notre base de données, ce qui empêche le modèle de les gérer correctement.

Comme attendu, l'augmentation de données permet d'obtenir de meilleurs résultats puisque notre base de données n'est pas très grande. De même pour le transfert d'apprentissage qui permet de rendre notre modèle efficace sur notre jeu de données malgré le faible nombre d'images.

CONCLUSION

Avec ce projet nous avons pu suivre toutes les étapes du choix du sujet à la récupération des données à la création et l'entraînement de modèles dessus et enfin à l'analyse des résultats. Ce projet nous a permis de mieux nous rendre compte de toutes les réflexions à avoir à chaque étape et des limites que nous avons pu rencontrer.

Parmi les principales difficultés, il y a notamment eu la constitution de la base de données qui a nécessité un tri important, ainsi que l'annotation qui s'est révélée fastidieuse et pas toujours uniforme entre les membres du groupe. On a également été confrontés aux limites d'un petit dataset, qui rend les résultats peu généralisables malgré nos efforts pour y remédier.

Pour avoir des performances meilleures, on pourrait augmenter la quantité d'images annotées dans la base de données (bien que cela prenne du temps), améliorer la précision des annotations avec des critères plus stricts et plus uniformes entre les différents 'annotateurs' et 'annotrices', et peut-être aussi pourrions nous utiliser des modèles plus grand et plus performants. On pourrait aussi augmenter la résolution d'entrée. En conclusion, il nous faudrait surtout beaucoup plus de temps d'annotations et d'entraînements de modèle.

On apprécie tout de même que notre projet nous a permis de travailler sur des images de films d'animation et nous a tous replongés en enfance!